SIMATS – Saveetha University

B.Tech – Artificial Intelligence and Machine Learning

Assignment 2 – CO2

Design C++ Programs by Applying Functions and Object-Oriented Concepts

Student Name	Appala Manohar
Register Number	192425360
Programme	B.Tech – Artificial Intelligence and Machine Learning
Institution	SIMATS – Saveetha University
Course	C++ Programming and Object-Oriented Programming
Course Outcome	CO2 – Functions and Object-Oriented Concepts
Academic Year	2026–2027
Assignment	Assignment 2
Repository Name	DSA0118
Repository Link	https://github.com/Appala-Manohar17/DSA0118_CPP
Compiler	g++ 13.3.0 (C++17, with -Wall -Wextra)

Q1 – Call by Value, by Reference and by Pointer

Program 1.1 – Concession Function Three Ways

Question: Write the hostel concession function three ways (by value, by reference, by pointer), predict the stored fee after each call, then verify.

Short Explanation: A fee of 45000 receives a 10% concession in three separate calls. The by-value function changes only its own copy, so the value printed inside looks correct but the stored fee does not change. The by-reference and by-pointer functions work on the caller's variable, so the stored fee really changes. The pointer version needs & at the call and * inside the function.

C++ Source Code:

```
#include <iostream>

void concessionByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
    std::cout << "inside by value     : " << fee << '\n';
}

void concessionByReference(long long& fee, int percent)
{
    fee -= fee * percent / 100;
    std::cout << "inside by reference : " << fee << '\n';
}

void concessionByPointer(long long* fee, int percent)
{
    *fee -= *fee * percent / 100;
    std::cout << "inside by pointer   : " << *fee << '\n';
}

int main()
{
    long long storedFee = 45000;

    concessionByValue(storedFee, 10);
    std::cout << "stored fee after value     : " << storedFee << '\n';

    concessionByReference(storedFee, 10);
    std::cout << "stored fee after reference : " << storedFee << '\n';

    concessionByPointer(&storedFee, 10);
    std::cout << "stored fee after pointer   : " << storedFee << '\n';
}
```

Program Output:

```
inside by value     : 40500
stored fee after value     : 45000
inside by reference : 40500
stored fee after reference : 40500
inside by pointer   : 36450
stored fee after pointer   : 36450
```

Code Screenshot:

C++ q1_three_ways.cpp

```cpp
#include <iostream>

void concessionByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
    std::cout << "inside by value     : " << fee << '\n';
}

void concessionByReference(long long& fee, int percent)
{
    fee -= fee * percent / 100;
    std::cout << "inside by reference : " << fee << '\n';
}

void concessionByPointer(long long* fee, int percent)
{
    *fee -= *fee * percent / 100;
    std::cout << "inside by pointer   : " << *fee << '\n';
}

int main()
{
    long long storedFee = 45000;

    concessionByValue(storedFee, 10);
    std::cout << "stored fee after value     : " << storedFee << '\n';

    concessionByReference(storedFee, 10);
    std::cout << "stored fee after reference : " << storedFee << '\n';

    concessionByPointer(&storedFee, 10);
    std::cout << "stored fee after pointer   : " << storedFee << '\n';
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task01$ g++ -std=c++17 -Wall -Wextra q1_three_ways.cpp -o q1_three_ways
~/Task01$ ./q1_three_ways
inside by value     : 40500
stored fee after value     : 45000
inside by reference : 40500
stored fee after reference : 40500
inside by pointer   : 36450
stored fee after pointer   : 36450
```

Prediction before running:

Call	Value printed inside	Predicted stored fee
By value (10%)	40500	45000 (unchanged)
By reference (10%)	40500	40500
By pointer (10% of 40500 = 4050)	36450	36450

The observed output matches all three predictions.

Inquiry:

- **By value:** the parameter fee is a new local variable in the called function's stack frame. It holds a copy of 45000, and it disappears when the function returns.
- **By reference:** no new fee variable is created. The parameter is another name for the caller's storedFee, so the function works on the caller's memory.
- **By pointer:** the function has a local pointer variable that holds the address of storedFee. The pointer is a copy, but *fee reaches the caller's variable.
- **Why only two change the caller's fee:** only the reference and the pointer reach the caller's original variable. The by-value function can only change its own copy.
- **When to use a const reference:** when the function only reads a large object (for example a Student record used to preview a fee). const Student& avoids copying the object, and the compiler blocks accidental changes.

Design Decision:

The function should return the new fee instead of changing it in place: fee = applyConcession(fee, percent). Several concessions then combine clearly, for example fee = applyConcession(applyConcession(fee, 10), 15). Each step is easy to test, a preview does not change the stored fee, and the assignment shows the change at the call. An in-place function hides the change and cannot preview. In-place is better only when the object is large and copying would be costly. The hostel office must also decide whether concessions stack one after another (10% then 15% gives 23.5% in total) or add up (25%); returning the new fee makes either rule easy to write.

Program 1.2 – Predict the Output

Question: Write the exact output of the given program before running it, then run it and explain every difference.

Short Explanation: This program mixes call by value, call by reference, call by pointer, return by reference and default arguments. Each of these decides which variables really

change. The output tells us which parameters were copies and which were the caller's own variables.

C++ Source Code:

```
#include <iostream>

void change(int a, int& b, int* c)
{
    a += 10;
    b += 10;
    *c += 10;
}

int& bigger(int& p, int& q) { return p > q ? p : q; }

void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }

int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';
    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}
```

Program Output:

```
1 12 13
1 100
16 13 6
```

Code Screenshot:

C++ q1_predict.cpp

```cpp
#include <iostream>

void change(int a, int& b, int* c)
{
    a += 10;
    b += 10;
    *c += 10;
}

int& bigger(int& p, int& q) { return p > q ? p : q; }

void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }

int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';
    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task01$ g++ -std=c++17 -Wall -Wextra q1_predict.cpp -o q1_predict
~/Task01$ ./q1_predict
1 12 13
1 100
16 13 6
```

Predicted output (derived by hand before running):

```
1 12 13
1 100
16 13 6
```

The observed output is identical, so there is no difference to explain.

- **change(a, b, &c):** a is a copy, so a stays 1. b is a reference to the caller's b, so it becomes 12. c is changed through its pointer, so it becomes 13. This prints 1 12 13.
- **bigger(a, c) = 100:** a is 1 and c is 13, so bigger returns a reference to c. The assignment writes 100 into c. This prints 1 100.

- **show(1):** y and z use their defaults 5 and 10, so 1 + 5 + 10 = 16.
- **show(1, 2):** only z uses its default 10, so 1 + 2 + 10 = 13.
- **show(1, 2, 3):** no default is used, so 1 + 2 + 3 = 6.
- Each show() prints a space after its number, so the last line ends with a space before the newline.

Program 1.3 – Complete the Code

Question: Complete TODO 1 so that the caller's fee is reduced by the percent using integer arithmetic (call by reference), and execute the program.

Short Explanation: The function applyConcession receives the fee by reference, so it can subtract the concession from the caller's own variable. The rest of the program is unchanged; only the TODO line and the comment lines were removed, because this document uses comment-free code. The expected output is 45000 and then 40500.

C++ Source Code:

```
#include <iostream>

void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
}

void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;
}

int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Program Output:

```
after call by value : 45000
after call by reference: 40500
```

Code Screenshot:

C++ q1_complete.cpp

```cpp
#include <iostream>

void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
}

void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;
}

int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task01$ g++ -std=c++17 -Wall -Wextra q1_complete.cpp -o q1_complete
~/Task01$ ./q1_complete
after call by value : 45000
after call by reference: 40500
```

Why the call by value leaves the fee at 45000: applyByValue works on a copy of hostelFee. The subtraction changes only that copy, and the copy is destroyed when the function returns, so hostelFee in main is never touched.

Alternative Concept and Complexity Analysis:

If the function returns the new fee, the solution is:

```
long long applyConcession(long long fee, int percent)
{
    return fee - fee * percent / 100;
}

hostelFee = applyConcession(hostelFee, 10);
```

The arithmetic takes O(1) time in every version. The difference is the data copied per call:

Passing style	long long (8 bytes)	Large Student object (S bytes)
By value	copies 8 bytes	copies S bytes every call: O(S) time and space
By reference	no copy of the value (only an address)	no copy of the object, O(1)
By const reference	same as reference, read-only	same as reference, read-only
Returned new value	returns 8 bytes	returns S bytes (copy, unless the compiler removes it)

Q2 – Return by Reference and Its Dangers

Program 2.1 – Returning a Reference to the Highest Mark

Question: Return a reference to the highest mark so a +2 moderation can be applied directly. Show use on the left of =, and compare returning a reference, a const reference and a copy.

Short Explanation: highestMark finds the position of the largest mark and returns the array element itself, not a copy. Because it returns a reference, the call can be used with += and on the left side of =. The read-only version returns a const reference and the copy version returns a plain int, which lets us compare the three styles.

C++ Source Code:

```
#include <iostream>

int& highestMark(int marks[], int size)
{
    int position = 0;
    for (int i = 1; i < size; i++)
    {
        if (marks[i] > marks[position])
            position = i;
    }
    return marks[position];
}

const int& highestMarkReadOnly(const int marks[], int size)
{
    int position = 0;
    for (int i = 1; i < size; i++)
    {
        if (marks[i] > marks[position])
            position = i;
    }
    return marks[position];
}

int highestMarkCopy(const int marks[], int size)
{
    int position = 0;
    for (int i = 1; i < size; i++)
    {
        if (marks[i] > marks[position])
            position = i;
    }
    return marks[position];
}

void printMarks(const int marks[], int size)
{
    for (int i = 0; i < size; i++)
        std::cout << marks[i] << ' ';
    std::cout << '\n';
}

int main()
{
```

```
    int marks[5] = {72, 88, 65, 91, 79};
    int size = 5;

    highestMark(marks, size) += 2;
    std::cout << "after +2 moderation : ";
    printMarks(marks, size);

    highestMark(marks, size) = 90;
    std::cout << "after call on left  : ";
    printMarks(marks, size);

    const int& readOnly = highestMarkReadOnly(marks, size);
    int copy = highestMarkCopy(marks, size);
    marks[3] = 100;
    std::cout << "const reference sees: " << readOnly << '\n';
    std::cout << "copy still holds    : " << copy << '\n';
}
```

Program Output:

```
after +2 moderation : 72 88 65 93 79
after call on left  : 72 88 65 90 79
const reference sees: 100
copy still holds    : 90
```

Code Screenshot:

C++ q2_return_reference.cpp part 1 of 2

```cpp
#include <iostream>

int& highestMark(int marks[], int size)
{
    int position = 0;
    for (int i = 1; i < size; i++)
    {
        if (marks[i] > marks[position])
            position = i;
    }
    return marks[position];
}

const int& highestMarkReadOnly(const int marks[], int size)
{
    int position = 0;
    for (int i = 1; i < size; i++)
    {
        if (marks[i] > marks[position])
            position = i;
    }
    return marks[position];
}

int highestMarkCopy(const int marks[], int size)
{
    int position = 0;
    for (int i = 1; i < size; i++)
    {
        if (marks[i] > marks[position])
            position = i;
```

C++ q2_return_reference.cpp part 2 of 2

```cpp
    }
    return marks[position];
}

void printMarks(const int marks[], int size)
{
    for (int i = 0; i < size; i++)
        std::cout << marks[i] << ' ';
    std::cout << '\n';
}

int main()
{
    int marks[5] = {72, 88, 65, 91, 79};
    int size = 5;

    highestMark(marks, size) += 2;
    std::cout << "after +2 moderation : ";
    printMarks(marks, size);

    highestMark(marks, size) = 90;
    std::cout << "after call on left  : ";
    printMarks(marks, size);

    const int& readOnly = highestMarkReadOnly(marks, size);
    int copy = highestMarkCopy(marks, size);
    marks[3] = 100;
    std::cout << "const reference sees: " << readOnly << '\n';
    std::cout << "copy still holds    : " << copy << '\n';
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task02$ g++ -std=c++17 -Wall -Wextra q2_return_reference.cpp -o q2_return_reference
~/Task02$ ./q2_return_reference
after +2 moderation : 72 88 65 93 79
after call on left  : 72 88 65 90 79
const reference sees: 100
copy still holds    : 90
```

Return type	What the caller can do	Observed result
int&	Read and change the array element; the call can be on the left of =	+2 changed 91 to 93; the assignment then set it to 90
const int&	Read only, but it is still an alias, so it sees later changes	Saw 100 after marks[3] = 100
int (copy)	Read only its own copy; later changes do not reach it	Still held 90

Program 2.2 – Returning a Reference to a Local Variable

Question: Return a reference to a local variable, show what the compiler warns about, and explain why using it is undefined behaviour.

Short Explanation: badReference returns a reference to localMark, which is destroyed as soon as the function ends. The compiler warns about this at compile time. Using the reference afterwards is undefined behaviour, so the program may crash, print garbage or appear to work.

C++ Source Code:

```
#include <iostream>

int& badReference()
{
    int localMark = 50;
    return localMark;
}

int main()
{
    int& dangling = badReference();
    std::cout << dangling << '\n';
}
```

Compiler and Program Output:

```
q2_dangling.cpp: In function ‘int& badReference()’:
q2_dangling.cpp:6:12: warning: reference to local variable ‘localMark’ returned [-
Wreturn-local-addr]
    6 |     return localMark;
      |            ^~~~~~~~~
q2_dangling.cpp:5:9: note: declared here
    5 |     int localMark = 50;
      |         ^~~~~~~~~
Segmentation fault
```

Code Screenshot:

C++ q2_dangling.cpp

```cpp
#include <iostream>

int& badReference()
{
    int localMark = 50;
    return localMark;
}

int main()
{
    int& dangling = badReference();
    std::cout << dangling << '\n';
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task02$ g++ -std=c++17 -Wall -Wextra q2_dangling.cpp -o q2_dangling
q2_dangling.cpp: In function ‘int& badReference()’:
q2_dangling.cpp:6:12: warning: reference to local variable ‘localMark’ returned
[-Wreturn-local-addr]
    6 |     return localMark;
      |            ^~~~~~~~~
q2_dangling.cpp:5:9: note: declared here
    5 |     int localMark = 50;
      |         ^~~~~~~~~
~/Task02$ ./q2_dangling
Segmentation fault
```

What the compiler warns about: the warning is ‘reference to local variable ‘localMark’ returned’ (-Wreturn-local-addr). It appears because the returned reference refers to an object that dies at the closing brace of the function.

Why using it is undefined behaviour: localMark lives in the stack frame of badReference, and that frame is released when the function returns. The reference now names an object that no longer exists. The C++ standard places no rule on what happens next, so the program may crash, print garbage or even print 50. In this run, g++ 13.3.0 produced a segmentation fault (exit code 139). A program that seems to work is still wrong.

Program 2.3 – Reference Invalidated When a Vector Grows

Question: The register is now a std::vector that may grow. Show why a reference taken before the vector grows may become invalid.

Short Explanation: A vector keeps its elements in one block of memory. When push_back needs more room, the vector allocates a bigger block, copies the elements and frees the old block. Any reference or pointer taken earlier still points into the freed block, so it is dangling.

C++ Source Code:

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> marks = {72, 88, 65};
    int* before = &marks[0];
    std::cout << "capacity before growth : " << marks.capacity() << '\n';

    marks.push_back(91);
    int* after = &marks[0];
    std::cout << "capacity after growth  : " << marks.capacity() << '\n';
    std::cout << "address unchanged      : " << std::boolalpha
              << (before == after) << '\n';

    int& fresh = marks[0];
    fresh += 2;
    std::cout << "first mark via new reference: " << marks[0] << '\n';
}
```

Program Output:

```
capacity before growth : 3
capacity after growth  : 6
address unchanged      : false
first mark via new reference: 74
```

Code Screenshot:

C++ q2_vector_growth.cpp

```cpp
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> marks = {72, 88, 65};
    int* before = &marks[0];
    std::cout << "capacity before growth : " << marks.capacity() << '\n';

    marks.push_back(91);
    int* after = &marks[0];
    std::cout << "capacity after growth  : " << marks.capacity() << '\n';
    std::cout << "address unchanged      : " << std::boolalpha
              << (before == after) << '\n';

    int& fresh = marks[0];
    fresh += 2;
    std::cout << "first mark via new reference: " << marks[0] << '\n';
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task02$ g++ -std=c++17 -Wall -Wextra q2_vector_growth.cpp -o q2_vector_growth
~/Task02$ ./q2_vector_growth
capacity before growth : 3
capacity after growth  : 6
address unchanged      : false
first mark via new reference: 74
```

Changed condition: the vector grew from capacity 3 to 6, so it moved its elements to a new block and freed the old one. The address of the first element changed, so any reference, pointer or iterator taken before the growth is now dangling. The safe pattern is to take the reference again after the vector has grown (as the last lines of the program do), or to keep an index instead.

Alternative Concept and Complexity Analysis:

If the highest mark is returned as an index or a pointer, the calling code looks like this:

```
int highestIndex(const int marks[], int size);
int* highestPointer(int marks[], int size);

marks[highestIndex(marks, size)] += 2;
*highestPointer(marks, size) += 2;
```

Return style	Time to find highest	Extra space	Safety if the array becomes a growing vector
int& (reference)	O(n)	O(1)	Dangling after growth; cannot say ‘no element’
int (index)	O(n)	O(1)	Still valid after growth (it names a position, not an address); can use -1 for none
int* (pointer)	O(n)	O(1)	Dangling after growth; can be nullptr for none, so the caller must check

All three scan the array once, so the time is O(n) and the space is O(1). The index is the safest choice when the register can grow.

Q3 – Default Arguments and Function Overloading

Program 3.1 – The Ambiguous Call

Question: Explain the compiler error produced by charge(double weight), charge(double weight, bool express) and charge(double weight, bool express = false, bool insured = false).

Short Explanation: The three functions overlap. A call with one double fits the first and the third function equally well, and a call with a double and a bool fits the second and the third equally well. The compiler cannot choose, so it reports an ambiguous call.

C++ Source Code:

```
#include <iostream>

double charge(double weight)
{
    return weight * 40;
}

double charge(double weight, bool express)
{
    return weight * 40 + (express ? 50 : 0);
}

double charge(double weight, bool express = false, bool insured = false)
{
    return weight * 40 + (express ? 50 : 0) + (insured ? 20 : 0);
}

int main()
{
    std::cout << charge(2.5) << '\n';
    std::cout << charge(2.5, true) << '\n';
}
```

Code Screenshot:

C++ q3_ambiguous.cpp

```cpp
#include <iostream>

double charge(double weight)
{
    return weight * 40;
}

double charge(double weight, bool express)
{
    return weight * 40 + (express ? 50 : 0);
}

double charge(double weight, bool express = false, bool insured = false)
{
    return weight * 40 + (express ? 50 : 0) + (insured ? 20 : 0);
}

int main()
{
    std::cout << charge(2.5) << '\n';
    std::cout << charge(2.5, true) << '\n';
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task03$ g++ -std=c++17 -Wall -Wextra q3_ambiguous.cpp -o q3_ambiguous
q3_ambiguous.cpp: In function ‘int main()’:
q3_ambiguous.cpp:20:24: error: call of overloaded ‘charge(double)’ is ambiguous
   20 |     std::cout << charge(2.5) << '\n';
      |                  ~~~~~~^~~~
q3_ambiguous.cpp:3:8: note: candidate: ‘double charge(double)’
    3 | double charge(double weight)
      |        ^~~~~~
q3_ambiguous.cpp:13:8: note: candidate: ‘double charge(double, bool, bool)’
   13 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
q3_ambiguous.cpp:21:24: error: call of overloaded ‘charge(double, bool)’ is ambiguous
   21 |     std::cout << charge(2.5, true) << '\n';
      |                  ~~~~~~^~~~~~~~~~
q3_ambiguous.cpp:8:8: note: candidate: ‘double charge(double, bool)’
    8 | double charge(double weight, bool express)
      |        ^~~~~~
q3_ambiguous.cpp:13:8: note: candidate: ‘double charge(double, bool, bool)’
   13 | double charge(double weight, bool express = false, bool insured = false)
      |        ^~~~~~
```

Explanation of the error: charge(2.5) matches charge(double) exactly, and it also matches charge(double, bool, bool) exactly once the defaults fill the missing arguments. Both are equally good, so the call is ambiguous. In the same way, charge(2.5, true) matches both the two-parameter function and the three-parameter function. The fix is to keep only one function for each kind of input and give that function the default arguments.

Program 3.2 – Redesigned charge Functions

Question: Design the charge functions with default arguments and overloading (grams as int, kg as double, a Parcel object) so that no call is ambiguous, and predict which function six calls select.

Short Explanation: There is now exactly one function for each kind of input: int grams, double kilograms and Parcel. Each has default arguments for express and insured, so common calls stay short. Because no two functions can match the same call equally well, no call is ambiguous.

C++ Source Code:

```
#include <iostream>

struct Parcel
{
    double weightKg;
    bool fragile;
};

double addOptions(double base, bool express, bool insured)
{
    if (express)
        base += 50;
    if (insured)
        base += 20;
    return base;
}

double charge(int weightGrams, bool express = false, bool insured = false)
{
    return addOptions(weightGrams / 1000.0 * 40, express, insured);
}

double charge(double weightKg, bool express = false, bool insured = false)
{
    return addOptions(weightKg * 40, express, insured);
}

double charge(const Parcel& parcel, bool express = false, bool insured = false)
{
    double base = parcel.weightKg * 40;
    if (parcel.fragile)
        base += 30;
    return addOptions(base, express, insured);
}

int main()
```

```
{
    std::cout << "charge(2)                    : " << charge(2) << '\n';
    std::cout << "charge(2.0f)                 : " << charge(2.0f) << '\n';
    std::cout << "charge(2.5, true)            : " << charge(2.5, true) << '\n';
    std::cout << "charge(2500, false, true)    : "
              << charge(2500, false, true) << '\n';
    std::cout << "charge(Parcel{3.0, false})   : "
              << charge(Parcel{3.0, false}) << '\n';
    std::cout << "charge(Parcel{1.5, true}, 1, 1): "
              << charge(Parcel{1.5, true}, true, true) << '\n';
}
```

Program Output:

```
charge(2)                    : 0.08
charge(2.0f)                 : 80
charge(2.5, true)            : 150
charge(2500, false, true)    : 120
charge(Parcel{3.0, false})   : 120
charge(Parcel{1.5, true}, 1, 1): 160
```

Code Screenshot:

C++ q3_redesign.cpp — part 1 of 2

```cpp
#include <iostream>

struct Parcel
{
    double weightKg;
    bool fragile;
};

double addOptions(double base, bool express, bool insured)
{
    if (express)
        base += 50;
    if (insured)
        base += 20;
    return base;
}

double charge(int weightGrams, bool express = false, bool insured = false)
{
    return addOptions(weightGrams / 1000.0 * 40, express, insured);
}

double charge(double weightKg, bool express = false, bool insured = false)
{
```

C++ q3_redesign.cpp part 2 of 2

```cpp
    return addOptions(weightKg * 40, express, insured);
}

double charge(const Parcel& parcel, bool express = false, bool insured = false)
{
    double base = parcel.weightKg * 40;
    if (parcel.fragile)
        base += 30;
    return addOptions(base, express, insured);
}

int main()
{
    std::cout << "charge(2)                    : " << charge(2) << '\n';
    std::cout << "charge(2.0f)                 : " << charge(2.0f) << '\n';
    std::cout << "charge(2.5, true)            : " << charge(2.5, true) << '\n';
    std::cout << "charge(2500, false, true)    : "
              << charge(2500, false, true) << '\n';
    std::cout << "charge(Parcel{3.0, false})   : "
              << charge(Parcel{3.0, false}) << '\n';
    std::cout << "charge(Parcel{1.5, true}, 1, 1): "
              << charge(Parcel{1.5, true}, true, true) << '\n';
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task03$ g++ -std=c++17 -Wall -Wextra q3_redesign.cpp -o q3_redesign
~/Task03$ ./q3_redesign
charge(2)                    : 0.08
charge(2.0f)                 : 80
charge(2.5, true)            : 150
charge(2500, false, true)    : 120
charge(Parcel{3.0, false})   : 120
charge(Parcel{1.5, true}, 1, 1): 160
```

Prediction before running and observed result:

Call	Function selected	Reason	Predicted	Observed
charge(2)	int grams	exact match for int	0.08	0.08
charge(2.0f)	double kg	float to double is a promotion; float to int is a conversion	80	80
charge(2.5, true)	double kg	exact match for double and bool	150	150
charge(2500, false, true)	int grams	exact match for int	120	120

charge(Parcel{3.0, false})	Parcel	exact match for Parcel	120	120
charge(Parcel{1.5, true}, true, true)	Parcel	exact match for Parcel	160	160

The value shows which function was chosen: charge(2) gives 0.08 because 2 was read as 2 grams, while charge(2.0f) gives 80 because 2.0 was read as 2 kg.

Inquiry:

- **How the compiler chooses:** it first lists the functions with the right name whose parameter count can match (defaults included). It then ranks each argument: an exact match is best, then a promotion (for example float to double or char to int), then a standard conversion (for example int to double or double to int), then a user-defined conversion. The function that is at least as good for every argument and better for one is chosen. If no single function wins, the call is ambiguous.
- **Why defaults go from the right:** arguments in a call are matched to parameters from the left, and a call can only leave out the last arguments. If a parameter in the middle had a default but a later one did not, a call could not skip the middle one. So once a parameter has a default, every parameter after it must also have one.

Alternative Concept and Complexity Analysis:

A single request structure with optional fields could replace the overloads:

```
struct ChargeRequest
{
    double weightKg;
    bool express = false;
    bool insured = false;
};

double charge(const ChargeRequest& request);
```

A function template (template <typename T> double charge(T weight)) could also share one body, but it cannot tell whether an int means grams or kilograms, so the meaning of the units would be lost.

Point	Overloads with default arguments	One request structure
Time	Resolved at compile time: no run-time cost, O(1)	No overload resolution; O(1)
Space copied per call	A few bytes (a number and two bools); a Parcel passed by const reference copies only an address	One const reference (an address)
Adding a new option	Add a parameter to every overload and re-check ambiguity	Add one field with a default; existing callers still compile

Q4 – Specifying a Class with an Array Inside It

Program 4.1 – SavingsAccount with a Transaction Array

Question: Specify SavingsAccount with private account number, balance, an array of 10 transactions and a count, and public deposit(), withdraw() and miniStatement(). Define some functions inside and some outside the class.

Short Explanation: The class keeps the last 10 transactions in a private array. deposit() is defined inside the class, while record(), withdraw() and miniStatement() are defined outside using the scope resolution operator. When the 11th transaction arrives, the array is shifted left by one place so the oldest transaction is discarded.

C++ Source Code:

```
#include <cmath>
#include <iomanip>
#include <iostream>

class SavingsAccount
{
    static const int MAX_TRANSACTIONS = 10;
    int accountNumber;
    double balance;
    double transactions[MAX_TRANSACTIONS];
    int count;

    void record(double amount);

public:
    SavingsAccount(int number, double openingBalance)
    {
        accountNumber = number;
        balance = openingBalance;
        count = 0;
    }

    void deposit(double amount)
    {
        if (amount <= 0)
            return;
        balance += amount;
        record(amount);
    }

    bool withdraw(double amount);
    void miniStatement() const;
};

void SavingsAccount::record(double amount)
{
    if (count == MAX_TRANSACTIONS)
    {
        for (int i = 0; i < MAX_TRANSACTIONS - 1; i++)
            transactions[i] = transactions[i + 1];
        transactions[MAX_TRANSACTIONS - 1] = amount;
    }
    else
    {
```

```
        transactions[count] = amount;
        count++;
    }
}

bool SavingsAccount::withdraw(double amount)
{
    if (amount <= 0 || amount > balance)
        return false;
    balance -= amount;
    record(-amount);
    return true;
}

void SavingsAccount::miniStatement() const
{
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Account " << accountNumber << " - last " << count
              << " transactions\n";
    for (int i = 0; i < count; i++)
    {
        std::cout << std::setw(2) << i + 1 << ". ";
        if (transactions[i] >= 0)
            std::cout << "Deposit  ";
        else
            std::cout << "Withdraw ";
        std::cout << std::setw(9) << std::fabs(transactions[i]) << '\n';
    }
    std::cout << "Balance: " << balance << '\n';
}

int main()
{
    SavingsAccount account(1001, 10000);

    account.deposit(5000);
    account.withdraw(1200);
    account.deposit(800);
    account.withdraw(300);
    account.deposit(2500);
    account.withdraw(700);
    account.deposit(1500);
    account.withdraw(400);
    account.deposit(900);
    account.withdraw(600);
    account.deposit(3000);
    account.withdraw(1000);

    if (!account.withdraw(100000))
        std::cout << "Withdrawal of 100000 refused: insufficient balance\n";

    account.miniStatement();
}
```

Program Output:

```
Withdrawal of 100000 refused: insufficient balance
Account 1001 - last 10 transactions
 1. Deposit     800.00
 2. Withdraw    300.00
 3. Deposit    2500.00
 4. Withdraw    700.00
 5. Deposit    1500.00
 6. Withdraw    400.00
 7. Deposit     900.00
 8. Withdraw    600.00
 9. Deposit    3000.00
10. Withdraw   1000.00
Balance: 19500.00
```

Code Screenshot:

C++ q4_savings_account.cpp — part 1 of 3

```cpp
#include <cmath>
#include <iomanip>
#include <iostream>

class SavingsAccount
{
    static const int MAX_TRANSACTIONS = 10;
    int accountNumber;
    double balance;
    double transactions[MAX_TRANSACTIONS];
    int count;

    void record(double amount);

public:
    SavingsAccount(int number, double openingBalance)
    {
        accountNumber = number;
        balance = openingBalance;
        count = 0;
    }

    void deposit(double amount)
    {
        if (amount <= 0)
            return;
        balance += amount;
        record(amount);
    }

    bool withdraw(double amount);
    void miniStatement() const;
};
```

C++ q4_savings_account.cpp part 2 of 3

```cpp

void SavingsAccount::record(double amount)
{
    if (count == MAX_TRANSACTIONS)
    {
        for (int i = 0; i < MAX_TRANSACTIONS - 1; i++)
            transactions[i] = transactions[i + 1];
        transactions[MAX_TRANSACTIONS - 1] = amount;
    }
    else
    {
        transactions[count] = amount;
        count++;
    }
}

bool SavingsAccount::withdraw(double amount)
{
    if (amount <= 0 || amount > balance)
        return false;
    balance -= amount;
    record(-amount);
    return true;
}

void SavingsAccount::miniStatement() const
{
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "Account " << accountNumber << " - last " << count
              << " transactions\n";
    for (int i = 0; i < count; i++)
    {
        std::cout << std::setw(2) << i + 1 << ". ";
```

C++ q4_savings_account.cpp part 3 of 3

```cpp
        if (transactions[i] >= 0)
            std::cout << "Deposit  ";
        else
            std::cout << "Withdraw ";
        std::cout << std::setw(9) << std::fabs(transactions[i]) << '\n';
    }
    std::cout << "Balance: " << balance << '\n';
}

int main()
{
    SavingsAccount account(1001, 10000);

    account.deposit(5000);
    account.withdraw(1200);
    account.deposit(800);
    account.withdraw(300);
    account.deposit(2500);
    account.withdraw(700);
    account.deposit(1500);
    account.withdraw(400);
    account.deposit(900);
    account.withdraw(600);
    account.deposit(3000);
    account.withdraw(1000);

    if (!account.withdraw(100000))
        std::cout << "Withdrawal of 100000 refused: insufficient balance\n";

    account.miniStatement();
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task04$ g++ -std=c++17 -Wall -Wextra q4_savings_account.cpp -o q4_savings_account
~/Task04$ ./q4_savings_account
Withdrawal of 100000 refused: insufficient balance
Account 1001 - last 10 transactions
 1. Deposit     800.00
 2. Withdraw    300.00
 3. Deposit    2500.00
 4. Withdraw    700.00
 5. Deposit    1500.00
 6. Withdraw    400.00
 7. Deposit     900.00
 8. Withdraw    600.00
 9. Deposit    3000.00
10. Withdraw   1000.00
Balance: 19500.00
```

Handling of the 11th transaction: twelve transactions were made, so the two oldest were discarded. The statement lists transactions 3 to 12 as lines 1 to 10. The withdrawal of 100000 was refused because it was larger than the balance, and it was not recorded. The final balance is 19500.00.

Inquiry:

- **Why the array must be private:** the class must keep the balance, the count and the stored transactions consistent. If the array were public, any code could overwrite a transaction or change the count and break the statement. Only deposit(), withdraw() and miniStatement() are allowed to change the data.
- **11th transaction with shifting:** the array is full, so the nine newest items move one place left and the new item goes in the last place. That is 9 copies for every new transaction, so the cost is O(k) for a history of k items.
- **11th transaction with a circular index:** the new item overwrites the oldest slot and the start index moves forward by one (wrapping at the end). That is one write, so the cost is O(1).

Design Decision (last 50 transactions):

The circular index changes least in effect. Both designs use the named constant MAX_TRANSACTIONS, so the number 10 is changed to 50 in one place. With shifting, however, every new transaction now moves 49 items instead of 9. With a circular index the cost stays one write. I would therefore switch record() to the circular index and keep the constant, the private data and the public functions the same.

Alternative Concept and Complexity Analysis:

Design	Time to add a transaction	Space used
Shifting fixed array (this program)	O(k) to shift k items	O(k), fixed array
Circular buffer with a start index	O(1)	O(k), fixed array plus one index
std::deque with push_back and pop_front	O(1) amortised	O(k) plus the deque's block bookkeeping and dynamic allocation

The circular buffer is measured against the shifting array in Q7.

Q5 – Static Members and Arrays of Objects

Program 5.1 – Predict the Output (Token)

Question: Write the exact output of the given Token program before running it, then run it and explain every difference.

Short Explanation: The static member next is shared by all Token objects and starts at 100. Each constructor call increases it by one and stores the result as the object's id. The static function issued() reports how many objects have been created.

C++ Source Code:

```
#include <iostream>

class Token {
    static int next;
    int id;
public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};

int Token::next = 100;

int main()
{
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get() << ' '
              << extra.get() << ' ' << Token::issued() << '\n';
}
```

Program Output:

```
101 103 104 4
```

Code Screenshot:

C++ q5_predict.cpp

```cpp
#include <iostream>

class Token {
    static int next;
    int id;
public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};

int Token::next = 100;

int main()
{
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get() << ' '
              << extra.get() << ' ' << Token::issued() << '\n';
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task05$ g++ -std=c++17 -Wall -Wextra q5_predict.cpp -o q5_predict
~/Task05$ ./q5_predict
101 103 104 4
```

Predicted output (derived by hand before running):

```
101 103 104 4
```

The observed output is identical, so there is no difference to explain.

Why the counter starts at 100 and how many times the constructor ran: the line int Token::next = 100; gives the single shared variable the value 100 before main starts. The constructor ran four times: three times for the array counter[3] and once for extra. The ids are therefore 101, 102, 103 and 104, and issued() returns 104 − 100 = 4.

Program 5.2 – Complete the Code (BusTicket)

Question: Complete TODO 1 and TODO 2 using the static members, and execute the program.

Short Explanation: TODO 1 gives a ticket the next seat number only while fewer than CAPACITY seats have been issued; otherwise the seat stays 0 and the ticket goes to the waiting list. TODO 2 returns the shared counter. The rest of the program is unchanged; only the TODO lines and comment lines were removed.

C++ Source Code:

```
#include <iostream>
#include <string>

class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        if (issued < CAPACITY)
        {
            issued++;
            seat = issued;
        }
    }
    static int count()
    {
        return issued;
    }
    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat)
                           : std::string("Waiting"))
                  << '\n';
    }
};

int BusTicket::issued = 0;

int main()
{
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Program Output:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

Code Screenshot:

C++ q5_bus_ticket.cpp — part 1 of 2

```cpp
#include <iostream>
#include <string>

class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        if (issued < CAPACITY)
        {
            issued++;
            seat = issued;
        }
    }
    static int count()
    {
        return issued;
```

C++ q5_bus_ticket.cpp — part 2 of 2

```cpp
    }
    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat)
                           : std::string("Waiting"))
                  << '\n';
    }
};

int BusTicket::issued = 0;

int main()
{
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task05$ g++ -std=c++17 -Wall -Wextra q5_bus_ticket.cpp -o q5_bus_ticket
~/Task05$ ./q5_bus_ticket
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

Inquiry:

- **Which members are static:** issued, CAPACITY and the function count(), because they belong to the whole bus, not to one ticket. seat and name are not static because every ticket has its own.

- **Where the static data member is defined:** the line inside the class is only a declaration. The definition int BusTicket::issued = 0; must appear once, outside the class, in one source file. CAPACITY is a static const int, so it can be given its value inside the class.
- **Why a static member function cannot use this:** it is not called on an object, so it has no this pointer and therefore cannot reach seat or name. It can only use static members such as issued.
- **How the array of objects is created:** BusTicket has no default constructor, so each element is initialised in the braces. The four constructors run in order, and the fourth ticket finds the bus full and gets seat 0 (Waiting).

Program 5.3 – Tickets Created Inside a Function Called Twice

Question: Create the tickets inside a function that is called twice and explain how the count behaves.

Short Explanation: bookGroup creates two BusTicket objects as local variables. They are destroyed when the function ends, but the static counter belongs to the class, so it keeps its value between the two calls.

C++ Source Code:

```
#include <iostream>
#include <string>

class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        if (issued < CAPACITY)
        {
            issued++;
            seat = issued;
        }
    }
    static int count()
    {
        return issued;
    }
    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat)
                           : std::string("Waiting"))
                  << '\n';
    }
};

int BusTicket::issued = 0;
```

```
void bookGroup(const std::string& first, const std::string& second)
{
    BusTicket tickets[2] = {BusTicket(first), BusTicket(second)};
    for (const BusTicket& t : tickets) t.print();
    std::cout << "Issued inside function: " << BusTicket::count() << '\n';
}

int main()
{
    std::cout << "First call\n";
    bookGroup("Asha", "Bala");
    std::cout << "Issued after first call: " << BusTicket::count() << '\n';

    std::cout << "Second call\n";
    bookGroup("Chitra", "Dev");
    std::cout << "Issued after second call: " << BusTicket::count() << '\n';
}
```

Program Output:

```
First call
Asha: Seat 1
Bala: Seat 2
Issued inside function: 2
Issued after first call: 2
Second call
Chitra: Seat 3
Dev: Waiting
Issued inside function: 3
Issued after second call: 3
```

Code Screenshot:

C++ q5_function_twice.cpp part 1 of 2

```cpp
#include <iostream>
#include <string>

class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        if (issued < CAPACITY)
        {
            issued++;
            seat = issued;
        }
    }
    static int count()
    {
        return issued;
    }
    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat)
```

C++ q5_function_twice.cpp part 2 of 2

```cpp
                            : std::string("Waiting"))
                  << '\n';
    }
};

int BusTicket::issued = 0;

void bookGroup(const std::string& first, const std::string& second)
{
    BusTicket tickets[2] = {BusTicket(first), BusTicket(second)};
    for (const BusTicket& t : tickets) t.print();
    std::cout << "Issued inside function: " << BusTicket::count() << '\n';
}

int main()
{
    std::cout << "First call\n";
    bookGroup("Asha", "Bala");
    std::cout << "Issued after first call: " << BusTicket::count() << '\n';

    std::cout << "Second call\n";
    bookGroup("Chitra", "Dev");
    std::cout << "Issued after second call: " << BusTicket::count() << '\n';
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task05$ g++ -std=c++17 -Wall -Wextra q5_function_twice.cpp -o q5_function_twice
~/Task05$ ./q5_function_twice
First call
Asha: Seat 1
Bala: Seat 2
Issued inside function: 2
Issued after first call: 2
Second call
Chitra: Seat 3
Dev: Waiting
Issued inside function: 3
Issued after second call: 3
```

Q6 – Friend Functions and Inline Functions

Program 6.1 – Complete the Code (Time and friend functions)

Question: Complete TODO 1 and TODO 2 inside the friend functions and execute the program.

Short Explanation: minutesBetween converts both times to minutes since midnight and returns the absolute difference. earlier compares the two minute values. Both are friend functions, so they can read the private members h and m of both objects.

C++ Source Code:

```
#include <iostream>

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;
    int difference = second - first;
    if (difference < 0)
        difference = -difference;
    return difference;
}

bool earlier(const Time& a, const Time& b)
{
    return a.h * 60 + a.m < b.h * 60 + b.m;
}

int main()
{
    Time departure(9, 45), arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure) << '\n';
}
```

Program Output:

```
205 false
```

Code Screenshot:

C++ q6_complete.cpp

```cpp
#include <iostream>

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;
    int difference = second - first;
    if (difference < 0)
        difference = -difference;
    return difference;
}

bool earlier(const Time& a, const Time& b)
{
    return a.h * 60 + a.m < b.h * 60 + b.m;
}

int main()
{
    Time departure(9, 45), arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure) << '\n';
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task06$ g++ -std=c++17 -Wall -Wextra q6_complete.cpp -o q6_complete
~/Task06$ ./q6_complete
205 false
```

Check by hand: 9:45 is 585 minutes and 13:10 is 790 minutes, so the difference is 205. The arrival is not earlier than the departure, so earlier(arrival, departure) is false. This matches the expected output 205 false.

Inquiry:

- **Why a friend function:** both times are equal partners in the calculation. A friend function takes both as ordinary arguments and can read the private members of both. A member function would favour one object (a.minutesTo(b)).
- **Friend versus member:** a friend is not part of the class. It is called without a dot and has no this pointer, but it can use the private members of the objects passed to it.
- **What inline asks the compiler to do:** it is a request to put the function body at the place of the call, which avoids the cost of a call. The compiler may ignore the request, and at higher optimisation levels it may inline functions that were not marked inline.
- **What is lost if the members are public:** encapsulation. Any code could set minutes to 75 or hours to 30 and break the meaning of a time.

Program 6.2 – Correctness Tests

Question: Test with times on either side of noon and the same time twice, and verify by hand.

Short Explanation: This program calls the friend functions on five pairs of times, including a pair on either side of noon, the same time twice and the extreme times of a day. The functions are marked inline because they are very small.

C++ Source Code:

```
#include <iostream>

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

inline int minutesBetween(const Time& a, const Time& b)
{
    int difference = (b.h * 60 + b.m) - (a.h * 60 + a.m);
    return difference < 0 ? -difference : difference;
}

inline bool earlier(const Time& a, const Time& b)
{
    return a.h * 60 + a.m < b.h * 60 + b.m;
}

void test(const char* label, const Time& a, const Time& b)
{
    std::cout << label << " -> minutes " << minutesBetween(a, b)
              << ", earlier(a, b) " << std::boolalpha << earlier(a, b)
              << ", earlier(b, a) " << earlier(b, a) << '\n';
}
```

```
int main()
{
    test("09:45 and 13:10 ", Time(9, 45), Time(13, 10));
    test("11:50 and 12:10 ", Time(11, 50), Time(12, 10));
    test("12:10 and 11:50 ", Time(12, 10), Time(11, 50));
    test("08:30 and 08:30 ", Time(8, 30), Time(8, 30));
    test("00:00 and 23:59 ", Time(0, 0), Time(23, 59));
}
```

Program Output:

```
09:45 and 13:10  -> minutes 205, earlier(a, b) true, earlier(b, a) false
11:50 and 12:10  -> minutes 20, earlier(a, b) true, earlier(b, a) false
12:10 and 11:50  -> minutes 20, earlier(a, b) false, earlier(b, a) true
08:30 and 08:30  -> minutes 0, earlier(a, b) false, earlier(b, a) false
00:00 and 23:59  -> minutes 1439, earlier(a, b) true, earlier(b, a) false
```

Code Screenshot:

C++ q6_tests.cpp

```cpp
#include <iostream>

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

inline int minutesBetween(const Time& a, const Time& b)
{
    int difference = (b.h * 60 + b.m) - (a.h * 60 + a.m);
    return difference < 0 ? -difference : difference;
}

inline bool earlier(const Time& a, const Time& b)
{
    return a.h * 60 + a.m < b.h * 60 + b.m;
}

void test(const char* label, const Time& a, const Time& b)
{
    std::cout << label << " -> minutes " << minutesBetween(a, b)
              << ", earlier(a, b) " << std::boolalpha << earlier(a, b)
              << ", earlier(b, a) " << earlier(b, a) << '\n';
}

int main()
{
    test("09:45 and 13:10 ", Time(9, 45), Time(13, 10));
    test("11:50 and 12:10 ", Time(11, 50), Time(12, 10));
    test("12:10 and 11:50 ", Time(12, 10), Time(11, 50));
    test("08:30 and 08:30 ", Time(8, 30), Time(8, 30));
    test("00:00 and 23:59 ", Time(0, 0), Time(23, 59));
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task06$ g++ -std=c++17 -Wall -Wextra q6_tests.cpp -o q6_tests
~/Task06$ ./q6_tests
09:45 and 13:10  -> minutes 205, earlier(a, b) true, earlier(b, a) false
11:50 and 12:10  -> minutes 20, earlier(a, b) true, earlier(b, a) false
12:10 and 11:50  -> minutes 20, earlier(a, b) false, earlier(b, a) true
08:30 and 08:30  -> minutes 0, earlier(a, b) false, earlier(b, a) false
00:00 and 23:59  -> minutes 1439, earlier(a, b) true, earlier(b, a) false
```

Check by hand: 11:50 is 710 minutes and 12:10 is 730 minutes, so the difference is 20 in both directions, and only the first time is earlier. The same time twice gives 0, and neither is earlier. 00:00 and 23:59 give 1439. All five lines match.

Program 6.3 – Timing 10,000,000 Calls

Question: Measure the time for 10,000,000 calls with and without inline, compiled with -O0 and with -O2.

Short Explanation: Four versions of the same calculation are timed: a friend function with the inline keyword, a friend function without it, a friend function forced to stay a real call, and a member function. The program is compiled and run twice, once with -O0 and once with -O2. The checksum proves that all versions calculate the same thing.

C++ Source Code:

```
#include <chrono>
#include <iostream>

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    int minutesTo(const Time& other) const
    {
        int difference = (other.h * 60 + other.m) - (h * 60 + m);
        return difference < 0 ? -difference : difference;
    }
    friend int minutesInline(const Time& a, const Time& b);
    friend int minutesPlain(const Time& a, const Time& b);
    friend int minutesNoInline(const Time& a, const Time& b);
};

inline int minutesInline(const Time& a, const Time& b)
{
    int difference = (b.h * 60 + b.m) - (a.h * 60 + a.m);
    return difference < 0 ? -difference : difference;
}

int minutesPlain(const Time& a, const Time& b)
{
    int difference = (b.h * 60 + b.m) - (a.h * 60 + a.m);
    return difference < 0 ? -difference : difference;
}

__attribute__((noinline)) int minutesNoInline(const Time& a, const Time& b)
{
    int difference = (b.h * 60 + b.m) - (a.h * 60 + a.m);
    return difference < 0 ? -difference : difference;
}

void report(const char* label, long long total,
            std::chrono::steady_clock::time_point start)
```

```
{
    auto end = std::chrono::steady_clock::now();
    double ms = std::chrono::duration<double, std::milli>(end - start).count();
    std::cout << label << ms << " ms   (checksum " << total << ")\n";
}

int main()
{
    const int CALLS = 10000000;
    long long total;
    std::chrono::steady_clock::time_point start;

    total = 0;
    start = std::chrono::steady_clock::now();
    for (int i = 0; i < CALLS; i++)
    {
        Time a(i % 24, i % 60);
        Time b((i + 7) % 24, (i + 13) % 60);
        total += minutesInline(a, b);
    }
    report("friend, inline keyword : ", total, start);

    total = 0;
    start = std::chrono::steady_clock::now();
    for (int i = 0; i < CALLS; i++)
    {
        Time a(i % 24, i % 60);
        Time b((i + 7) % 24, (i + 13) % 60);
        total += minutesPlain(a, b);
    }
    report("friend, no keyword     : ", total, start);

    total = 0;
    start = std::chrono::steady_clock::now();
    for (int i = 0; i < CALLS; i++)
    {
        Time a(i % 24, i % 60);
        Time b((i + 7) % 24, (i + 13) % 60);
        total += minutesNoInline(a, b);
    }
    report("friend, forced call    : ", total, start);

    total = 0;
    start = std::chrono::steady_clock::now();
    for (int i = 0; i < CALLS; i++)
    {
        Time a(i % 24, i % 60);
        Time b((i + 7) % 24, (i + 13) % 60);
        total += a.minutesTo(b);
    }
    report("member minutesTo       : ", total, start);
}
```

Program Output (both optimisation levels):

```
-O0:
friend, inline keyword : 81.5087 ms   (checksum 5954164188)
friend, no keyword     : 62.653 ms   (checksum 5954164188)
friend, forced call    : 57.0497 ms   (checksum 5954164188)
member minutesTo       : 57.6911 ms   (checksum 5954164188)
-O2:
friend, inline keyword : 25.6817 ms   (checksum 5954164188)
friend, no keyword     : 25.5779 ms   (checksum 5954164188)
friend, forced call    : 27.3702 ms   (checksum 5954164188)
member minutesTo       : 26.2185 ms   (checksum 5954164188)
```

Code Screenshot:

C++ q6_timing.cpp — part 1 of 3

```cpp
#include <chrono>
#include <iostream>

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    int minutesTo(const Time& other) const
    {
        int difference = (other.h * 60 + other.m) - (h * 60 + m);
        return difference < 0 ? -difference : difference;
    }
    friend int minutesInline(const Time& a, const Time& b);
    friend int minutesPlain(const Time& a, const Time& b);
    friend int minutesNoInline(const Time& a, const Time& b);
};

inline int minutesInline(const Time& a, const Time& b)
{
    int difference = (b.h * 60 + b.m) - (a.h * 60 + a.m);
    return difference < 0 ? -difference : difference;
}

int minutesPlain(const Time& a, const Time& b)
{
    int difference = (b.h * 60 + b.m) - (a.h * 60 + a.m);
    return difference < 0 ? -difference : difference;
}

__attribute__((noinline)) int minutesNoInline(const Time& a, const Time& b)
```

C++ q6_timing.cpp — part 2 of 3

```cpp
{
    int difference = (b.h * 60 + b.m) - (a.h * 60 + a.m);
    return difference < 0 ? -difference : difference;
}

void report(const char* label, long long total,
            std::chrono::steady_clock::time_point start)
{
    auto end = std::chrono::steady_clock::now();
    double ms = std::chrono::duration<double, std::milli>(end - start).count();
    std::cout << label << ms << " ms   (checksum " << total << ")\n";
}

int main()
{
    const int CALLS = 10000000;
    long long total;
    std::chrono::steady_clock::time_point start;

    total = 0;
    start = std::chrono::steady_clock::now();
    for (int i = 0; i < CALLS; i++)
    {
        Time a(i % 24, i % 60);
        Time b((i + 7) % 24, (i + 13) % 60);
        total += minutesInline(a, b);
    }
    report("friend, inline keyword : ", total, start);

    total = 0;
```

C++ q6_timing.cpp part 3 of 3

```cpp
    start = std::chrono::steady_clock::now();
    for (int i = 0; i < CALLS; i++)
    {
        Time a(i % 24, i % 60);
        Time b((i + 7) % 24, (i + 13) % 60);
        total += minutesPlain(a, b);
    }
    report("friend, no keyword     : ", total, start);

    total = 0;
    start = std::chrono::steady_clock::now();
    for (int i = 0; i < CALLS; i++)
    {
        Time a(i % 24, i % 60);
        Time b((i + 7) % 24, (i + 13) % 60);
        total += minutesNoInline(a, b);
    }
    report("friend, forced call    : ", total, start);

    total = 0;
    start = std::chrono::steady_clock::now();
    for (int i = 0; i < CALLS; i++)
    {
        Time a(i % 24, i % 60);
        Time b((i + 7) % 24, (i + 13) % 60);
        total += a.minutesTo(b);
    }
    report("member minutesTo       : ", total, start);
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task06$ g++ -std=c++17 -Wall -Wextra -O0 q6_timing.cpp -o q6_timing_O0
~/Task06$ ./q6_timing_O0
friend, inline keyword : 81.5087 ms   (checksum 5954164188)
friend, no keyword     : 62.653 ms   (checksum 5954164188)
friend, forced call    : 57.0497 ms   (checksum 5954164188)
member minutesTo       : 57.6911 ms   (checksum 5954164188)
~/Task06$ g++ -std=c++17 -Wall -Wextra -O2 q6_timing.cpp -o q6_timing_O2
~/Task06$ ./q6_timing_O2
friend, inline keyword : 25.6817 ms   (checksum 5954164188)
friend, no keyword     : 25.5779 ms   (checksum 5954164188)
friend, forced call    : 27.3702 ms   (checksum 5954164188)
member minutesTo       : 26.2185 ms   (checksum 5954164188)
```

Growth Investigation – measured results (10,000,000 calls):

Version	-O0 (ms)	-O0 range, 4 runs	-O2 (ms)	-O2 range, 4 runs
friend, inline keyword	81.5	56.9 – 81.5	25.7	25.7 – 30.4
friend, no keyword	62.7	58.5 – 62.7	25.6	25.6 – 32.1
friend, forced call	57.0	57.0 – 59.0	27.4	27.4 – 32.3
member minutesTo	57.7	57.7 – 63.8	26.2	26.2 – 30.3

The times were measured in the environment used to prepare this document (g++ 13.3.0, Ubuntu) and will differ on another computer, so run the program on your own machine and use your own numbers when you submit.

- **At -O0** GCC does not inline any function, even with the inline keyword, so all four versions make real calls. The differences between them are within the run-to-run noise shown in the range column.
- **At -O2** the compiler inlines the small functions in the same file whether or not the keyword is present. The forced-call version is only slightly slower, because the modulo operations that create the test times take most of the loop time. The cost of one call is a small fraction of a nanosecond here.
- The optimisation level matters far more than the keyword: the -O2 times are roughly half of the -O0 times. inline is a request that the compiler may ignore.

Deployment Decision:

The friend functions should stay. The comparison is symmetric, the friend version is as fast as the member version in the measurements, and only these two named functions get access to the private members. Public getters would expose h and m to all code. If many more functions need access, the better change is to add member functions or a small public interface instead of a growing friend list.

Alternative Concept and Complexity Analysis:

Design	Time per call	Encapsulation
Friend functions (this program)	O(1); same as member after inlining	Private data stays hidden; only the named friends can read it
Member functions a.minutesTo(b), a.isEarlier(b)	O(1); measured same as friend at -O2	Private data stays hidden; call syntax favours object a
Public getters used by ordinary functions	O(1); same after inlining	Weakest: h and m are readable by all code

Q7 – Diagnose, Redesign and Validate

Selected program: the SavingsAccount transaction history from Q4 (the shifting array).

A. Diagnose the Limitation

The Q4 program works well for 10 transactions, but the requirement can change (the bank already asked for 50). In ShiftingLog, which uses the same record() algorithm, the time for 500,000 additions grew from 2.94 ms for k = 10 to 109.69 ms for k = 5000 (measured in Program 7.1). The cause is the design: the oldest transaction must always sit at index 0, so once the log is full every addition moves k − 1 items. The cost per addition is O(k).

B. Define the Design Objective

For 500,000 additions into a history of k = 5000 transactions, the redesigned log must be at least 10 times faster than the shifting log, must hold exactly the same transactions (equal checksum), and must still list them oldest first.

C. Design the Revised Solution

The shifting array is replaced by a circular buffer. A private index start marks the oldest transaction. When the log is not full, the new item goes to position (start + count) % capacity. When it is full, the new item overwrites the oldest slot and start moves forward by one, wrapping around. at(position) returns items[(start + position) % capacity], so the list is still read oldest first. No extra array is needed, only one extra int (start). The trade-offs are that the stored order is no longer the time order, the index arithmetic is harder to read, and each addition pays for a modulo operation.

Pseudocode:

```
add(amount):
    if count < capacity:
        items[(start + count) % capacity] = amount
        count = count + 1
    else:
        items[start] = amount
        start = (start + 1) % capacity

at(position):
    return items[(start + position) % capacity]
```

Class diagram:

```
+-----------------------------------+
| CircularLog                       |
+-----------------------------------+
| - items    : vector<long long>    |
| - capacity : int                  |
| - start    : int                  |
| - count    : int                  |
+-----------------------------------+
| + add(amount)                     |
| + at(position) : long long        |
+-----------------------------------+
```

D. Predict Before Implementing

Prediction (written before running the benchmark): the circular log will take about the same time for every k, because each addition is one write. The shifting log will grow roughly in proportion to k. At k = 10 the two will differ by less than 2 times, and at k = 5000 the circular log will be at least 100 times faster. The measure is the total time in milliseconds for 500,000 additions.

Program 7.1 – Original and Redesigned Transaction Log

Question: Compare the original shifting array with the redesigned circular buffer using the same tests and measurements.

Short Explanation: ShiftingLog is the original design from Q4 (shift every element left once the log is full). CircularLog is the redesign that keeps a start index and overwrites the oldest slot. Both receive 500,000 transactions for four sizes of history, and a checksum of the final contents proves that both keep identical data.

C++ Source Code:

```
#include <chrono>
#include <iomanip>
#include <iostream>
#include <vector>

class ShiftingLog
{
    std::vector<long long> items;
    int capacity;
    int count;
public:
    explicit ShiftingLog(int size) : items(size), capacity(size), count(0) {}

    void add(long long amount)
    {
        if (count == capacity)
        {
            for (int i = 0; i < capacity - 1; i++)
                items[i] = items[i + 1];
            items[capacity - 1] = amount;
```

```
        }
        else
        {
            items[count] = amount;
            count++;
        }
    }

    long long at(int position) const { return items[position]; }
};

class CircularLog
{
    std::vector<long long> items;
    int capacity;
    int start;
    int count;
public:
    explicit CircularLog(int size)
        : items(size), capacity(size), start(0), count(0) {}

    void add(long long amount)
    {
        if (count < capacity)
        {
            items[(start + count) % capacity] = amount;
            count++;
        }
        else
        {
            items[start] = amount;
            start = (start + 1) % capacity;
        }
    }

    long long at(int position) const
    {
        return items[(start + position) % capacity];
    }
};

template <typename Log>
long long checksum(const Log& log, int size)
{
    long long total = 0;
    for (int i = 0; i < size; i++)
        total += log.at(i) * (i + 1);
    return total;
}

int main()
{
    const int OPERATIONS = 500000;
    const int sizes[4] = {10, 50, 500, 5000};

    std::cout << std::fixed << std::setprecision(2);
    std::cout << "  last-k   shifting(ms)   circular(ms)   speed-up   same\n";

    for (int size : sizes)
    {
        ShiftingLog shifting(size);
        CircularLog circular(size);

        auto begin = std::chrono::steady_clock::now();
        for (int i = 0; i < OPERATIONS; i++)
            shifting.add(i % 1000 + 1);
        auto middle = std::chrono::steady_clock::now();
```

```
        for (int i = 0; i < OPERATIONS; i++)
            circular.add(i % 1000 + 1);
        auto finish = std::chrono::steady_clock::now();

        std::chrono::duration<double, std::milli> shiftingTime = middle - begin;
        std::chrono::duration<double, std::milli> circularTime = finish - middle;
        double shiftingMs = shiftingTime.count();
        double circularMs = circularTime.count();
        bool same = checksum(shifting, size) == checksum(circular, size);

        std::cout << std::setw(8) << size << std::setw(15) << shiftingMs
                  << std::setw(15) << circularMs << std::setw(10)
                  << shiftingMs / circularMs << "x   "
                  << (same ? "yes" : "NO") << '\n';
    }
}
```

Program Output:

```
  last-k   shifting(ms)   circular(ms)   speed-up   same
      10           2.94           2.23      1.32x   yes
      50           4.81           2.25      2.14x   yes
     500          14.22           2.42      5.88x   yes
    5000         109.69           2.42     45.36x   yes
```

Code Screenshot:

C++ q7_benchmark.cpp — part 1 of 3

```cpp
#include <chrono>
#include <iomanip>
#include <iostream>
#include <vector>

class ShiftingLog
{
    std::vector<long long> items;
    int capacity;
    int count;
public:
    explicit ShiftingLog(int size) : items(size), capacity(size), count(0) {}

    void add(long long amount)
    {
        if (count == capacity)
        {
            for (int i = 0; i < capacity - 1; i++)
                items[i] = items[i + 1];
            items[capacity - 1] = amount;
        }
        else
        {
            items[count] = amount;
            count++;
        }
    }

    long long at(int position) const { return items[position]; }
};

class CircularLog
{
    std::vector<long long> items;
    int capacity;
```

C++ q7_benchmark.cpp part 2 of 3

```cpp
    int start;
    int count;
public:
    explicit CircularLog(int size)
        : items(size), capacity(size), start(0), count(0) {}

    void add(long long amount)
    {
        if (count < capacity)
        {
            items[(start + count) % capacity] = amount;
            count++;
        }
        else
        {
            items[start] = amount;
            start = (start + 1) % capacity;
        }
    }

    long long at(int position) const
    {
        return items[(start + position) % capacity];
    }
};

template <typename Log>
long long checksum(const Log& log, int size)
{
    long long total = 0;
    for (int i = 0; i < size; i++)
        total += log.at(i) * (i + 1);
    return total;
}

```

C++ q7_benchmark.cpp — part 3 of 3

```cpp
int main()
{
    const int OPERATIONS = 500000;
    const int sizes[4] = {10, 50, 500, 5000};

    std::cout << std::fixed << std::setprecision(2);
    std::cout << "  last-k   shifting(ms)   circular(ms)   speed-up   same\n";

    for (int size : sizes)
    {
        ShiftingLog shifting(size);
        CircularLog circular(size);

        auto begin = std::chrono::steady_clock::now();
        for (int i = 0; i < OPERATIONS; i++)
            shifting.add(i % 1000 + 1);
        auto middle = std::chrono::steady_clock::now();
        for (int i = 0; i < OPERATIONS; i++)
            circular.add(i % 1000 + 1);
        auto finish = std::chrono::steady_clock::now();

        std::chrono::duration<double, std::milli> shiftingTime = middle - begin;
        std::chrono::duration<double, std::milli> circularTime = finish - middle;
        double shiftingMs = shiftingTime.count();
        double circularMs = circularTime.count();
        bool same = checksum(shifting, size) == checksum(circular, size);

        std::cout << std::setw(8) << size << std::setw(15) << shiftingMs
                  << std::setw(15) << circularMs << std::setw(10)
                  << shiftingMs / circularMs << "x   "
                  << (same ? "yes" : "NO") << '\n';
    }
}
```

Output Screenshot:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
~/Task07$ g++ -std=c++17 -Wall -Wextra -O2 q7_benchmark.cpp -o q7_benchmark
~/Task07$ ./q7_benchmark
  last-k   shifting(ms)   circular(ms)   speed-up   same
      10           2.94           2.23      1.32x   yes
      50           4.81           2.25      2.14x   yes
     500          14.22           2.42      5.88x   yes
    5000         109.69           2.42     45.36x   yes
```